

Chapter 4

Higher Inductive Types

Coquand et al. (2018) は、cubical type theory を高次帰納型 (*higher inductive types*) へ拡張する方法を与えた。その鍵が、まさに Section 3.3–Section 3.4 で採った「合成を hcomp と transp に分ける」提示である：高次帰納型では、合成構造を「構成子としての hcomp 」と「構成子ごとに与える transp の κ -簡約」として指定できる。

4.1 A Common Pattern for Higher Inductive Types

CHM の考える高次帰納型の例はすべて共通のパターンに従う。これは cubical type theory における高次帰納型の構文的スキーマの定式化への第一歩とみなせるが、スキーマの正確な定式化とその意味論的対応物は future work とされている。

各高次帰納型 $D(\vec{z} : \vec{P})$ は、(ambient context Γ 上の) パラメータの telescope $\vec{z} : \vec{P}$ と構成子のリスト $\vec{c}$ で指定される*¹。各構成子 c は次のデータで指定される：

$$c : (\vec{x} : \vec{A}(\vec{z})) [\vec{i}] D(\vec{z}) \quad \text{境界データ } \varphi(\vec{i}) \mapsto e(\vec{z}, \vec{x}, \vec{i})$$

ここで telescope $\vec{x} : \vec{A}$ は c の引数の型を指定する (propositional truncation のような再帰的高次帰納型では、 D 自身が $\vec{A}$ に現れてよい)。名前 $\vec{i}$ の長さは、 c が導入する立方体の次元を指定する：長さが 0, 1, 2 のとき、 c をそれぞれ point 構成子、path 構成子、square 構成子と呼ぶ。境界データ $\varphi \mapsto e$ は構成子 c の端点を指定する： φ は自由変数が $\vec{i}$ に含まれる $\mathbb{F}$ の元であり、 e は部分元

$$\vec{z} : \vec{P}, \vec{x} : \vec{A}(\vec{z}), \vec{i} : \mathbb{I}, \varphi(\vec{i}) \vdash e(\vec{z}, \vec{x}, \vec{i}) : D(\vec{z})$$

であって、リスト $\vec{c}$ の中で先行する構成子（および後述の hcomp ）のみに言及するものである。

telescope $\vec{z} : \vec{P}$ の各インスタンス $\vec{u} : \vec{P}$ に対して $D(\vec{u})$ は型であり、上のように指定された構成子 c の導入規則は (ambient context を省略して) 次で与えられる：

$$\frac{\vec{v} : \vec{A}(\vec{u}) \quad \vec{r} : \mathbb{I}}{c \vec{v} \vec{r} : D(\vec{u})}$$

さらに $\varphi(\vec{r}) = 1_{\mathbb{F}}$ が成り立つ場合には、判断的等値性

$$c \vec{v} \vec{r} =_{\partial} e(\vec{u}, \vec{v}, \vec{r}) : D(\vec{u})$$

*¹ 文脈 Γ 上の telescope $x_1 : A_1, \dots, x_n : A_n$ ($\vec{x} : \vec{A}$ と書く) とは、 $\Gamma, \vec{x} : \vec{A}$ が well-formed な文脈となるような (空でもよい) 対象変数宣言のリストである。したがって $\vec{x} : \vec{A}$ は文脈制限 Δ, φ も名前宣言 $i : \mathbb{I}$ も含まない。

が成り立つ (Remark 35 の ∂ -等値性にあたる)。この判断的等値性のため、関数 $f : (x : D(\vec{u})) \rightarrow P(x)$ を定義するときには、 f がこの等値性を保存すること、すなわち

$$\varphi(\vec{r}) \vdash f(c \vec{v} \vec{r}) = f(e(\vec{u}, \vec{v}, \vec{r})) : P(c \vec{v} \vec{r})$$

が保証されなければならない。この要件は $D(\vec{u})$ の除去規則 (eliminator) の型付け規則で扱われるべきものであるが、その一般的定式化は (「extension types」に類する拡張を要するため) future work とされている。

Section 3.3 冒頭の見取り図で予告した第三の場合が、まさに高次帰納型である：path 構成子があるため、側面は構成子頭とは限らず、hcomp の κ -簡約は与えられない。そこで homogeneous composition の構造は「構成子として」導入される。といっても、新しい規則は要らない。 $A := D(\vec{u})$ に対する hcomp の型付けと ∂ -等値性は、Definition 27 がそのまま与える ($D(\vec{u})$ は Γ で型付けされ、方向 i に依存しない—homogeneous composition が適用できるのはこのためである。 i は側面 v の中には自由に現れてよい)。高次帰納型に固有なのは、次の 2 点だけである。第一に、この hcomp には κ -簡約を与えない。したがって $\text{hcomp}_{D(\vec{u})}^i [\varphi \mapsto v] v_0$ という形の元それ自体が $D(\vec{u})$ の標準形となる (暗黙の構成子とみなしてよい)。第二に、その代償として、 $D(\vec{u})$ からの除去写像には hcomp との可換性を述べる計算規則が付く： $f : D(\vec{u}) \rightarrow C$ に対し $f(\text{hcomp}_{D(\vec{u})}^i [\varphi \mapsto v] v_0) \rightarrow_{\beta} \text{hcomp}_C^i [\varphi \mapsto f(v)] (f(v_0))$ の形の規則である (Chapter 5 の $\mathbb{Z}$ の除去規則を参照。依存除去の一般形は上述のとおり future work に属する)。以下の例では、この hcomp 構成子を毎回書かないが、考える各高次帰納型の定義にはつねに含まれているものとする。

Section 3.3 の見取り図の第二の場合で触れた「hcomp を残す設計」とは、自然数型のような通常の帰納型にこの扱いを適用することにあたる：hcomp を再帰で消去する代わりに、構成子 $\text{hcomp}_{\mathbb{N}}^i$ を持たせてもよい。この「弱い」形の自然数型は通常のものと同値であり、したがって (univalence により) 等しいことが証明できる。

移送については、Section 3.4 の transp がそのまま用いられる： $\Gamma, i : \mathbb{I} \vdash \vec{u} : \vec{P}$ が与えられたとき、線 $A := D(\vec{u})$ に対する $\text{transp}^i A \varphi u_0$ である。ただしその κ -簡約は、Section 3.4 のような型の形ごとの再帰では与えられず、 u_0 の取りうる形ごとに与えられる：すなわち構成子 c ごとに 1 つ、そして hcomp 構成子に対して 1 つである。後者は

$$\begin{aligned} & \text{transp}^i A \varphi (\text{hcomp}_{A[0/i]}^j [\psi \mapsto u] u_0) \\ & \rightarrow_{\kappa} \text{hcomp}_{A[1/i]}^j [\psi \mapsto \text{transp}^i A \varphi u] (\text{transp}^i A \varphi u_0) \end{aligned}$$

で与えられる (i と j はともに束縛変数なので、名前替えにより $i \neq j$ と仮定してよい)。hcomp の場合はすべての例で同じ形なので、以下では各高次帰納型の transp の定義からは省略する。

Section 3.4 の comp の定義は、hcomp と transp (と squeeze) のみを用いていたことを思い出そう。したがって、高次帰納型の合成は、「構成子に適用された場合の transp」を与えるだけで定まる。

4.2 The Circle

共通パターンの最初の例として、最も単純な高次帰納型である円周 (circle) S^1 を見よう。

Definition 41 (The Circle S^1). 高次帰納型 S^1 は、パラメータなしで、次の2つの構成子により定義される：

$$\frac{\Gamma \text{ context}}{\Gamma \vdash \text{base} : S^1} \quad \frac{\Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \text{loop } r : S^1}$$

ただし loop は次の境界の等値性判断 (∂ -等値性) をともなう：

$$\Gamma \vdash \text{loop } 0 =_{\partial} \text{base} : S^1 \quad \Gamma \vdash \text{loop } 1 =_{\partial} \text{base} : S^1$$

(共通パターンに従い、構成子 $\text{hcomp}_{S^1}^i$ も定義に含まれる。)

base は point 構成子、 loop は path 構成子であり、 $\langle i \rangle \text{loop } i : \text{Path } S^1 \text{ base base}$ を与える。すなわち S^1 は、1つの点と、その点から自分自身への1本の path とからなる型である。名前のとおり、これは円周を表す。

ここで、等号型 (Section 3.5) との違いが鮮明になる。等号型のもとで UIP を仮定すれば、 base から base への証明はすべて $\text{refl}(\text{base})$ に等しいことになり、 loop は 1_{base} と区別できない。しかし Path 型のもとでは両者は区別され、実際 $\text{Path}(\text{Path } S^1 \text{ base base}) 1_{\text{base}} (\langle i \rangle \text{loop } i)$ は inhabited でないことが証明できる*2。これが Section 3.5 で述べた「型は集合ではなく高次の構造をもちうる」ことの最も簡単な実例である。

S^1 はパラメータをもたないから、線が定数の場合しか生じず、 transp の κ -簡約は自明である：

$$\text{transp}^i S^1 \varphi u_0 \rightarrow_{\kappa} u_0$$

したがって S^1 上の合成もすべて homogeneous であり、 hcomp だけで足りる。

型 C への関数 $f : S^1 \rightarrow C$ を定義するには、点 $c : C$ とループ $l : \text{Path } C c c$ を与えればよく、 $f(\text{base}) \rightarrow_{\beta} c$ 、 $f(\text{loop } r) \rightarrow_{\beta} l r$ 、および hcomp との可換性が成り立つ。依存版 (motive $P : S^1 \rightarrow \text{type}$) では、 loop に対する証明義務は P 自身が loop に沿って動くため、Section 2.3 の依存 path

$$\bar{l} : \text{PathP}^i P(\text{loop } i) \bar{c} \bar{c} \quad (\bar{c} : P(\text{base}) \text{ に対して})$$

となる。これが PathP を導入した理由である。

4.3 The Torus

Section 4.1 のスキーマは、長さ2の名前リストをもつ構成子—square 構成子—を許すが、ここまでの例はいずれも長さ1 (path 構成子) であった。square 構成子の代表的な実例がトーラス T^2 である。

*2 S^1 の基本群が $\mathbb{Z}$ であること ($\pi_1(S^1) = \mathbb{Z}$) の帰結である。homotopy type theory における証明は The Univalent Foundations Program (2013) の第8章を、cubical type theory での構成的な扱いは Mörtberg (2021) を参照。本書では立ち入らない。

Definition 42 (The Torus T^2). 高次帰納型 T^2 は、パラメータなしで、次の構成子により定義される ($k = 1, 2$) :

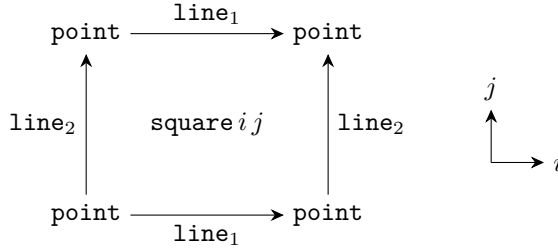
$$\frac{\Gamma \text{ context}}{\Gamma \vdash \text{point} : T^2} \quad \frac{\Gamma \vdash r : \mathbb{I}}{\Gamma \vdash \text{line}_k r : T^2} \quad \frac{\Gamma \vdash r : \mathbb{I} \quad \Gamma \vdash s : \mathbb{I}}{\Gamma \vdash \text{square } r s : T^2}$$

ただし line_k と square は次の境界の等値性判断 (∂ -等値性) をともなう :

$$\begin{aligned} \Gamma \vdash \text{line}_k 0 =_{\partial} \text{point} : T^2 & \quad \Gamma \vdash \text{line}_k 1 =_{\partial} \text{point} : T^2 \\ \Gamma \vdash \text{square } 0 s =_{\partial} \text{line}_2 s : T^2 & \quad \Gamma \vdash \text{square } 1 s =_{\partial} \text{line}_2 s : T^2 \\ \Gamma \vdash \text{square } r 0 =_{\partial} \text{line}_1 r : T^2 & \quad \Gamma \vdash \text{square } r 1 =_{\partial} \text{line}_1 r : T^2 \end{aligned}$$

(共通パターンに従い、構成子 $\text{hcomp}_{T^2}^i$ も定義に含まれる。)

square の境界の 4 本の等値性は、対辺を貼り合わせた正方形というトーラスの標準的な描像をそのまま読み上げたものである : 下辺 ($j = 0$) と上辺 ($j = 1$) がともに line_1 、左辺 ($i = 0$) と右辺 ($i = 1$) がともに line_2 である。



同じ型を等号型の語彙で与えると、次のようになる (The Univalent Foundations Program (2013) §6.6) :

$$\text{point} : T^2 \quad p, q : \text{point} =_{T^2} \text{point} \quad \text{surf} : p \bullet q = q \bullet p$$

ここで注目すべきは、最後の構成子を述べるために path の合成 $\bullet$ (本書の $p \cdot q$ にあたる、J から導出される演算) が必要になることである。型の定義が導出済みの演算に依存し、しかも $p \bullet q$ という書き方には合成の結合の向き of 選択が入り、2 本の辺は非対称に (一方は等式の左辺、他方は右辺に) 現れる。cubical のスキーマでは、 square は単に区間変数を 2 つもつ項であり、その境界を 4 本の ∂ -等値性で対等に読み上げるだけでよい — 導出された演算も J も現れない。 n 次元の構成子は n 個の区間変数をもつ項である、という形で、次元が上がっても枠組みは変わらないのである。

この記法上の差は、証明の労力の差として現れる。たとえば $T^2 \simeq S^1 \times S^1$ は、等号型の語彙では独立の論文を要する仕事だったが (Sojakova による 2016 年の ACM TOCL 論文)、cubical type theory では大幅に短い証明が知られている。なお依存除去では、 square の像の型が 2 つの区間変数の両方に依存するため、証明義務は依存 path (Section 2.3) の 2 次元版 — PathP の入れ子 — となる。また T^2 はパラメータをもたないから、 S^1 と同様、 transp の κ -簡約は自明であり、 T^2 上の合成はすべて homogeneous である。

History and Further Reading

高次帰納型は homotopy type theory の文脈で導入され (circle・suspension・truncation・商型などの例とともに HoTT book (The Univalent Foundations Program, 2013) の第 6 章で展開された)、その意味論は Lumsdaine and Shulman (2020) により与えられた。cubical type theory における高次帰納型の構成的な扱いが Coquand et al. (2018) (本章の転記元) であり、合成を `hcomp` と `trans` (本書の `transp`) に分解するその手法は、cartesian 側では Cavallo and Harper (2019) が対応する。

Bibliography

- Altenkirch, T., C. McBride, and W. Swierstra. (2007) “Observational equality, now!”, In *Proceedings of the 2007 Workshop on Programming Languages meets Program Verification (PLPV '07)*, A. Stump and H. Xi (eds.). pp.57–68.
- Altenkirch, T. and L. Scoccola. (2020) “The Integers as a Higher Inductive Type”, In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.67–73, ACM.
- Angiuli, C., G. Brunerie, T. Coquand, R. Harper, K.-B. Hou (Favonia), and D. R. Licata. (2021) “Syntax and models of Cartesian cubical type theory”, *Mathematical Structures in Computer Science* **31**(4), pp.424–468.
- Angiuli, C., R. Harper, and T. Wilson. (2017) “Computational higher-dimensional type theory”, In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. pp.680–693, ACM.
- Awodey, S. and M. A. Warren. (2009) “Homotopy theoretic models of identity types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **146**(1), pp.45–55.
- Bekki, D. (2014) “Representing Anaphora with Dependent Types”, In *Proceedings of Logical Aspects of Computational Linguistics (8th international conference, LACL2014, Toulouse, France), LNCS 8535*, N. Asher and S. V. Soloviev (eds.). pp.14–29, Springer, Heidelberg.
- Bekki, D. and K. Mineshima. (2017) “Context-Passing and Underspecification in Dependent Type Semantics”, In: S. Chatzikyriakidis and Z. Luo (eds.): *Modern Perspectives in Type-Theoretical Semantics*, Vol. 98 of *Studies in Linguistics and Philosophy*. Springer, pp.11–41.
- Bezem, M., T. Coquand, and S. Huber. (2014) “A Model of Type Theory in Cubical Sets”, In *Proceedings of 19th International Conference on Types for Proofs and Programs (TYPES 2013)*, R. Matthes and A. Schubert (eds.), Vol. 26 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.107–128, Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- Cavallo, E. and R. Harper. (2019) “Higher Inductive Types in Cubical Computational Type Theory”, *Proceedings of the ACM on Programming Languages* **3**(POPL), pp.1:1–1:27.
- Cohen, C., T. Coquand, S. Huber, and A. Mörtberg. (2018) “Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom”, In *Proceedings of 21st International Conference on Types for Proofs and Programs (TYPES 2015)*, T. Uustalu (ed.), Vol. 69 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.5:1–5:34, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Preprint: arXiv:1611.02108.
- Coquand, T., S. Huber, and A. Mörtberg. (2018) “On Higher Inductive Types in Cubical Type Theory”, In *Proceedings of LICS '18: Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. pp.255–264.

-
- Hofmann, M. (1995) “Extensional concepts in intensional type theory”, Ph.D. thesis, University of Edinburgh.
- Hofmann, M. and T. Streicher. (1998) “The groupoid interpretation of type theory”, In: G. Sambin and J. M. Smith (eds.): *Twenty-Five Years of Constructive Type Theory (Venice, 1995)*, Oxford Logic Guides, vol.36. New York, Oxford University Press.
- Huber, S. (2019) “Canonicity for Cubical Type Theory”, *Journal of Automated Reasoning* **63**(2), pp.173–210.
- Kapulkin, K. and P. L. Lumsdaine. (2021) “The simplicial model of Univalent Foundations (after Voevodsky)”, *Journal of the European Mathematical Society* **23**(6), pp.2071–2126.
- Lumsdaine, P. L. and M. Shulman. (2020) “Semantics of higher inductive types”, *Mathematical Proceedings of the Cambridge Philosophical Society* **169**(1), pp.159–208.
- Martin-Löf, P. (1975) “An intuitionistic theory of types”, In: H. E. Rose and J. Shepherdson (eds.): *Logic Colloquium ’73*. Amsterdam, North-Holland, pp.73–118.
- Martin-Löf, P. (1984) *Intuitionistic Type Theory*, Vol. 17. Naples, Italy: Bibliopolis. Sambin, Giovanni (ed.).
- Mörtberg, A. (2021) “Cubical methods in homotopy type theory and univalent foundations”, *Mathematical Structures in Computer Science* **31**(10), pp.1147–1184.
- Orton, I. and A. M. Pitts. (2016) “Axioms for Modelling Cubical Type Theory in a Topos”, In Proceedings of *25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, J.-M. Talbot and L. Regnier (eds.), Vol. 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*. pp.24:1–24:19, Schloss Dagstuhl – Leibniz-Zentrum für Informatik. Journal version: *Logical Methods in Computer Science* 14(4:23), 2018.
- Sterling, J. and C. Angiuli. (2021) “Normalization for Cubical Type Theory”, In Proceedings of *36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2021)*. pp.1–15, IEEE.
- Streicher, T. (1993) “Investigations into Intensional Type Theory”, Ph.D. thesis, Habilitationsschrift, Ludwig-Maximilians-Universität.
- The Univalent Foundations Program. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*. <https://homotopytypetheory.org/book>.
- Vezzosi, A., A. Mörtberg, and A. Abel. (2019) “Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types”, *Proceedings of the ACM on Programming Languages* **3**, pp.87:1–87:29.